03
Data Structures in Functional Programming
organization, expressiveness, lazyness

Mirko Viroli
mirko.viroli@unibo.it

C.D.L. Magistrale in Ingegneria e Scienze Informatiche
Alma Mater Studiorum—Università di Bologna, Cesena

a.a. 2025/2026

Outline

1. Generics
2. On pragmatical expressiveness
3. Declarativity and Lambda-calculus
4. Evaluation order
5. Streams
6. Extension Methods and OO-style method calls

Generics

Parametric polymorphism (see e.g. Java generics)

- enhancing re-usability of code, and expressiveness of types
- abstracting some code (functions, types, classes) from some type

In Scala

- types, case classes, methods, enums, can be generic
- syntax `[A,B,..]` is used to express generics
- type variables can have type bounds (e.g., `A <: B`)
- inference of types typically works very well

OptionalInts

```scala
// overall module
object OptionalInts:

  // type "public" definition, exposing structure
  enum OptionalInt:
    case Just(i: Int)
    case Empty

  // operations (/algorithms)
  object OptionalInt:

    def isEmpty(opt: OptionalInt): Boolean = opt match
      case Empty => true
      case _     => false

    def orElse(opt: OptionalInt, orElse: Int): Int = opt match
      case Just(a) => a
      case _       => orElse

    def map(opt: OptionalInt)(f: Int => Int): OptionalInt = opt match
      case Just(a) => Just(f(a))
      case _       => Empty
```

Generifying: Optionals

La notazione definisce i parametri di tipo generici del metodo e stabilisce una relazione gerarchica tra di essi tramite un vincolo di tipo (type bound):
- "A" è il tipo del valore contenuto nell'opzionale di partenza (Optional[A]).
- "B >: A" è una Lower Type Bound (vincolo di tipo inferiore). In Scala, l'operatore ">:" si legge "è un supertipo di". Indica che il parametro di tipo B deve essere uguale ad A oppure un suo supertipo (una classe padre o un'interfaccia/trait più generale).

```scala
// overall module
object Optionals:

  // type "public" definition, exposing structure
  enum Optional[A]:
    case Just(a: A)
    case Empty() // here parens are needed because of genericity

  // operations (/algorithms)
  object Optional:

    def isEmpty[A](opt: Optional[A]): Boolean = opt match
      case Empty() => true
      case _       => false

    def orElse[A, B >: A](opt: Optional[A], orElse: B): B = opt match
      case Just(a) => a
      case _       => orElse

    def map[A, B](opt: Optional[A])(f: A => B): Optional[B] = opt match
      case Just(a) => Just(f(a))
      case _       => Empty()
```

Optionals: tests in a main

```scala
@main def tryOptionals =
  import Optionals.* // to work with Optionals (to see Optional type)
  import Optional.* // to directly access algorithms

  val s1: Optional[Int] = Just(1)
  val s2: Optional[Int] = Empty()

  println(s1) // Some(1)
  println(isEmpty(s1)) // false
  println(orElse(s1, 0)) // 1
  println(orElse(s2, 0)) // 0
  println(map(s1)(i => "val: " + 1)) // Some("val: 1")
  println(map(s2)(i => "val: " + 1)) // None
```

Optionals: JUnit serious test

```
1  class OptionTest:
2
3    // imports needed to work with Options
4    import u03.Optionals.*
5    import Optional.*
6
7    // values under test
8    val optSome = Just(5)
9    val optNone = Empty()
10
11   @Test def testEmpty() =
12     assertFalse(isEmpty(optSome))
13     assertTrue(isEmpty(optNone))
14
15   @Test def testOrElse() =
16     assertEquals(5, orElse(optSome, 10))
17     assertEquals(10, orElse(optNone, 10))
18
19   @Test def testMap() =
20     assertEquals(Just(6), map(optSome)(_ + 1))
21     assertEquals(Just("val: 5"), map(optSome)(v => "val: " + v))
```

Optionals: JUnit serious test, improved a bit

```scala
class OptionTestImproved:

  // imports needed to work with Options
  import u03.Optionals.*
  import Optional.*

  // values under test
  val optVal = 5
  val optSome = Just(optVal)
  val optNone = Empty()

  @Test def testEmpty() =
    assertFalse(isEmpty(optSome))
    assertTrue(isEmpty(optNone))

  @Test def testOrElse() =
    val otherVal = 10
    assertEquals(optVal, orElse(optSome, otherVal))
    assertEquals(otherVal, orElse(optNone, otherVal))

  @Test def testMap() =
    assertEquals(Just(optVal + 1), map(optSome)(_ + 1))
    assertEquals(Just("val: " + optVal), map(optSome)(v => "val: " + v))
    assertEquals(Empty(), map(optNone)(v => "val: " + v))
```

Generics: methods and case classes

```scala
object Generics extends App :

  // Generic methods, in 1 and 2 type variables
  def f1[A](a: A, b: A): A = a
  def f2[A, B](a: A, b: B): A = a

  // Calling a generic methods may infer type variables
  println(f1(10, 20)) // 10
  println(f2("10", 20)) // "10"
  println(f2[String, Int]("10", 20)) // "10"

  // Generic case class, in 2 type variables
  case class Pair[A, B](a: A, b: B)
  // Defining a value with type Pair[Int, String]: can omit type
    variables
  val p = Pair(10, "20")
  // Defining a generic method to operate on pairs
  def swap[A, B](p: Pair[A, B]): Pair[B, A] = p match
    case Pair(a, b) => Pair(b, a)

  println(swap(p)) // Pair("20", 10)
```

The case of generic tuples (and built-in tuples)

```scala
object Tuples extends App :

  case class Tup1[A](a: A)
  case class Tup2[A, B](a: A, b: B)
  case class Tup3[A, B, C](a: A, b: B, c: C)
  case class Tup4[A, B, C, D](a: A, b: B, c: C, d: D)

  val tu = Tup2(10, true)
  val tu2: Tup2[Int, Boolean] = Tup2(10, true)

  println(s"$tu, $tu2, ${tu == tu2}") // Tup2(10,true), Tup2(10,true), true

  // Built-in tuples: Tuple2, Tuple3, .... Tuple22, aliased to (,) (,,)
     (,,,)
  val t = (10, true)
  val t2 = Tuple2(10, true)

  println(s"$t, $t2, ${t == t2}") // (10,true), (10,true), true
```

Outline

1. Generics
2. On pragmatical expressiveness
3. Declarativity and Lambda-calculus
4. Evaluation order
5. Streams
6. Extension Methods and OO-style method calls

Powerful algebraic data types in FP

Significa suddividere i parametri di una funzione in più liste separate di argomenti (anziché un'unica lista), separando i parametri che definiscono come eseguire un'operazione (la configurazione o la strategia) da quelli che rappresentano i dati da elaborare.

Design

- product/sum type definition on top (`enum` or `case class`)
  - typically, structure is exposed, ~~hence~~ così algorithms can be implemented outside
  - some part of the structure may be defined as private
- an object with algorithmic implementation aspects
  - methods for operations and for construction altogether
- generics are useful to extend reusability, by abstraction
- higher-order functions allow ~~a great deal~~ un notevole livello di of factorisation/reuse
- configuration/strategy arguments often grouped by currying

$\Rightarrow$ are there some domain concepts you cannot address this way?

Key examples

- Optional, Sequences (Lists), Trees, Streams

Sequences

```scala
object Sequences: // Essentially, generic linkedlists

  enum Sequence[E]:
    case Cons(head: E, tail: Sequence[E])
    case Nil()

  object Sequence:

    def sum(l: Sequence[Int]): Int = l match
      case Cons(h, t) => h + sum(t)
      case _          => 0

    def map[A, B](l: Sequence[A])(mapper: A => B): Sequence[B] = l match
      case Cons(h, t) => Cons(mapper(h), map(t)(mapper))
      case Nil()      => Nil()

    def filter[A](l1: Sequence[A])(pred: A => Boolean): Sequence[A] = l1 match
      case Cons(h, t) if pred(h) => Cons(h, filter(t)(pred))
      case Cons(_, t)            => filter(t)(pred)
      case Nil()                 => Nil()

@main def trySequences =
  import Sequences.*
  val l = Sequence.Cons(10, Sequence.Cons(20, Sequence.Cons(30, Sequence.Nil())))
  println(Sequence.sum(l)) // 30

  import Sequence.*

  println(sum(map(filter(l)(_ >= 20))(_ + 1))) // 21+31 = 52
```

Dati da elaborare con map

Strategia da usare per map

rende visibile ADT e suo companion object

Sequences: tests

```scala
import org.junit.*
import org.junit.Assert.*

class SequenceTest:
  import u03.Sequences.*
  import Sequence.*

  val l: Sequence[Int] = Cons(10, Cons(20, Cons(30, Nil())))

  @Test def testSum() =
    assertEquals(0, sum(Nil()))
    assertEquals(60, sum(l))

  @Test def testMap() =
    assertEquals(Cons(11, Cons(21, Cons(31, Nil()))), map(l)(_ + 1))
    assertEquals(Cons("10", Cons("20", Cons("30", Nil()))), map(l)(_ + ""))

  @Test def testFilter() =
    assertEquals(Cons(20, Cons(30, Nil())), filter(l)(_ >= 20))
    assertEquals(Cons(10, Cons(30, Nil())), filter(l)(_ != 20))
```

On algegraic data-types, classes and information hiding

Algebraic data-types vs classes

- classes introduce types, giving constructors/operations and hiding internal state
- algebraic data-types give constructors/operations, but do not hide internal state Normalmente le varianti di un enum (quindi la sua struttura) sono pubbliche.
- namely, you are exposed with the structure of data (`enum` and `cases`)

Encapsulating/hiding state with Scala

- declare your cases as `private`
- add construction operations in the `object`
- from outside, nobody will be able to pattern match tramite il metodo 'unapply' nel companion object)
- (will later see how this could be circumvented)

Sequences encapsulating (some) implementation

```scala
object EncapsulatedSequences:

  // A generic sequence with encapsulated state
  enum Sequence[E]:
    private case Cons(head: E, tail: Sequence[E])
    private case Nil()

  // cases are visible only here
  object Sequence:
    def cons[E](head: E, tail: Sequence[E]): Sequence[E] = Cons(head, tail)

    def nil[E](): Sequence[E] = Nil[E]()

    def sum(l: Sequence[Int]): Int = l match
      case Cons(h, t) => h + sum(t)
      case _          => 0

    def map[A, B](l: Sequence[A])(mapper: A => B): Sequence[B] = l match
      case Cons(h, t) => Cons(mapper(h), map(t)(mapper))
      case Nil()      => Nil()

object AdditionalSequencesAlgorithms:
  import EncapsulatedSequences.*
  // cases are not accessible: how could this be implemented?
  def filter[A](l1: Sequence[A])(pred: A => Boolean): Sequence[A] = ???

@main def tryEncapsulatedSequences =
  import EncapsulatedSequences.*, Sequence.*

  val l = cons(10, cons(20, cons(30, nil())))
  println(sum(map(l)(_ + 1)))
```

Per poter eseguire il pattern matching su una variante (ad esempio case Cons(head, tail)), il compilatore deve poter accedere alla dichiarazione di quel caso specifico e ai suoi estrattori. Quando un caso viene marcato come private case, la sua definizione diventa accessibile esclusivamente all'interno del modulo/file sorgente in cui è stato definito.

i due costruttori vanno nel companion object

Outline

1. Generics
2. On pragmatical expressiveness
3. Declarativity and Lambda-calculus
4. Evaluation order
5. Streams
6. Extension Methods and OO-style method calls

Declaration vs Procedure

Differenza tra approccio procedurale (focalizzato sui passaggi per ottenere un risultato: come eseguire il calcolo passo dopo passo) e dichiarativo (focalizzato sul cosa calcolare anziché sul come: non viene specificato l'ordine esatto delle singole istruzioni). L'astrazione dall'ordine di esecuzione rende la programmazione funzionale particolarmente adatta all'esecuzione parallela e distribuita.

Consider the map/filter/sum specification

- `sum(map(filter(l)(_>=20))(_+1))`
- does it mention what is performed, and in which order?
- no, it instead focusses on building a sequence out of other sequences, then aggregate by _+1

Poiché il codice dichiara solo le trasformazioni dei dati e non impone un flusso procedurale rigido, il motore di esecuzione ha la massima libertà di scegliere come valutare l'espressione. Il risultato finale rimane identico indipendentemente da dove o come l'espressione viene calcolata: su un singolo core oppure su un processore multi-core sfruttando il parallellismo oppure in un ambiente di rete distribuito su centinaia di macchine.

Consider parallelism/concurrency

- a functional expression could be evaluated by a concurrent system (a PC with multi-core architecture) or by a distributed system (many resources in the cloud)
- the functional expression (and result) is the same, what happens "inside" changes significantly
- actually, a high number of modern distributed platforms (Apache Spark, Map-Reduce, and so on), are essentially FP over the so called *cloud-edge continuum*

L'infrastruttura sottostante cambia radicalmente l'esecuzione interna per ottimizzare le prestazioni, ma la correttezza del risultato è garantita dalla natura della funzione.

Deepen declarativity

Informal definition

- a specification is said to be declarative, if it focusses on what has to be achieved, not how it should be achieved
- in a programming context: a program is declarative if it abstracts from the step-by-step behaviour needed to achieve the result
- in a sense, the program expresses more the intended result, rather than the process to obtain it—a sort of *holy grail* of computer science...

Declarativity of functional programming

- in FP: a program is a (possibly very complex) composition of functions, e.g.: $a(b(c(1,2),3),d(4))$
- which function application we do first? should we care?
- intuition: if $f(x,y) = x + x$, we could compute $f(g(),h())$ in various ways:
  - compute $g()$ and $h()$, get results a and b, and then compute $a + a$
  - reduce to $g() + g()$, then reduce to $a + g()$, then to $a + a$
  - compute $g()$ to a, then directly compute $a + a$

  tutti e tre giungono allo stesso identico valore finale

  L'ordine con cui le sotto-espressioni vengono valutate diventa quindi irrilevante per la correttezza del programma.
- in certain cases we expect order of evaluation as being irrelevant

Declarativity: problem and method

Tramite il lambda-calcolo si dimostra formalmente la proprietà della confluenza: essa prova che, qualunque sia l'ordine delle semplificazioni/riduzioni applicate a un'espressione, il risultato finale, se esiste ed è raggiungibile, sarà sempre unico e identico.

Problems statement

- which functional programs are independent of evaluation order?
- how can we control evaluation order when needed?

Un approccio formale per rispondere a questi interrogativi in modo scientifico

~~A formal approach~~, rooted in a foundation of programming languages

- define a fully-expressive subset of FP, namely, the λ-calculus
- derive a result of so-called "confluence", i.e., "don't care nondeterminism" (proof omitted[a])
- define the concept of "order of evaluations" on the λ-calculus (not developed here)
- note: historically, λ-calculus (1930) grounded computability, FP, and later OOP...
- λ-calculus and Turing machines are the backbone of whole computer science

[a] `https://plfa.inf.ed.ac.uk/Confluence/`

Key ideas of λ-calculus

A very very small language with:

- one-argument functions
- function application (applying a function to an expression)
- programs are functional compositions of expressions
- many-argument functions, primitive types, product types, sum types, recursion . . . (it is shown they are just derived concepts)

352 A. CHURCH.

An occurrence of a variable **x** in a given formula is called an occurrence of **x** as a *bound variable* in the given formula if it is an occurrence of **x** in a part of the formula of the form λ**x**[**M**]; that is, if there is a formula **M** such that λ**x**[**M**] occurs in the given formula and the occurrence of **x** in question is an occurrence in λ**x**[**M**]. All other occurrences of a variable in a formula are called occurrences as a *free variable*.

A formula is said to be *well-formed* if it is a variable, or if it is one of the symbols Π, Σ, &, $\sim$, ι, A, or if it is obtainable from these symbols by repeated combinations of them of one of the forms {**M**}(**N**) and λ**x**[**M**], where **x** is any variable and **M** and **N** are symbols or formulas which are being combined. This is a definition by induction. It implies the following rules: (1) a variable is well-formed (2) Π, Σ, &, $\sim$, ι, and A are well-formed (3) if **M** and **N** are well-formed then {**M**}(**N**) is well-formed (4) if **x** is a variable and **M** is well-formed then λ**x**[**M**] is well-formed.

Lambda Calculus (A.Church, 1930)

Syntax of λ-expressions

$$L ::= x \mid (\lambda x.L) \mid L_1 L_2$$

- x stands for any "variable" (i.e., any function formal argument)
- $\lambda x.L$ stands for lambda with argument x and body L (Scala's "x=>L")
- $L_1 L_2$ stands applying L_1 to arg L_2 (Scala's "$L_1(L_2)$")

Conventions

- application is left-associative: $L_1 L_2 L_3 \equiv (L_1 L_2) L_3$
- abstraction is right-associative: $\lambda x.\lambda y.L \equiv \lambda x.(\lambda y.L)$
- application has higher priority wrt abstraction: $\lambda x.L_1 L_2 \equiv \lambda x.(L_1 L_2)$
- $L_e\{L_P/x\}$ is substitution of any occurrence of x with L_P into L_e

Transition, also called "(beta) reduction" (applicable at any level of nesting)

$$(\lambda x.L_B) L_P \longrightarrow L_B\{L_P/x\}$$

- "calling a function gives the body L_B after substituting formal parameter x with actual L_P", i.e., the semantics of "passing an argument"

Lambda Calculus in Scala style

An untyped subset of Scala

$$V ::= x \mid y \mid z \mid \ldots \quad L ::= V \mid V \texttt{=>} L \mid L_1(L_2)$$

Example programs (with a suitable definition of tpe L)

- `val t: L = x => y => x` as a lambda: $\lambda x.\lambda y.x$
- `val f: L = x => y => y` as a lambda: $\lambda x.\lambda y.y$
- `val n: L = b => b(f)(t)` as a lambda: $\lambda b.b(\lambda x.\lambda y.y)(\lambda x.\lambda y.x)$
- `val a: L = b1 => b2 => b1(b2)(f)` as a lambda: $\lambda b1.\lambda b2.b1\ b2(\lambda x.\lambda y.y)$
- `val o: L = b1 => b2 => b1(t)(b2)` as a lambda: $\lambda b1.\lambda b2.b1(\lambda x.\lambda y.x)b2$

Some results

- `t("a")("b")` clearly gives `"a"` $(\lambda x.\lambda y.x)ab \to (\lambda y.a)b \to a$
- `f("a")("b")` clearly gives `"b"` $(\lambda x.\lambda y.y)ab \to (\lambda y.y)b \to b$
- `n(t)`, `n(f)`?
- `a(f)(f)`, `a(f)(t)`, `a(t)(f)`, `a(t)(t)`?
- what about `o`?

Church encoding of booleans

Building data types

$\Rightarrow$ defining some values as constructors

$\Rightarrow$ defining some values as functions to manipulate constructors

Encoding booleans

- $T := \lambda x.\lambda y.x$, $F := \lambda x.\lambda y.y$
- $Not := \lambda x.xFT$, $And := \lambda x.\lambda y.xyF$

Sequences of reductions (any similar sequence leads to the same result!)

$$
\begin{aligned}
&And(Not\ F)T \equiv And(\underline{(\lambda x.xFT)F})T \rightarrow \\
&And(FFT)T \equiv And(\underline{(\lambda x.\lambda y.y)FT})T \rightarrow\rightarrow \\
&And\,TT \equiv \underline{(\lambda x.\lambda y.xyF)TT} \rightarrow\rightarrow \\
&TTF \equiv \underline{(\lambda x.\lambda y.x)TF} \rightarrow\rightarrow \\
&T \not\rightarrow
\end{aligned}
$$

Lambda-based boolean support in Scala

```scala
object Lambda:

  // Define a type Lambda that extends type Lambda => Lambda itself
  // Just one of the many magic tricks we will see of Scala
  trait Lambda extends (Lambda => Lambda)

  val True: Lambda = x => y => x
  val False: Lambda = x => y => y
  val Not: Lambda = b => b(False)(True)
  val And: Lambda = b1 => b2 => b1(b2)(False)
  val Or: Lambda = b1 => b2 => b1(True)(b2)
```

```scala

abstract class LambdaTest:
  import Lambda.*

  def toBoolean(l: Lambda): Boolean =
    val l1: Lambda = x => x
    val l2: Lambda = y => y
    l(l1)(l2) match
      case res if res == l1 => true
      case res if res == l2 => false
      case _ => ??? // throws an Exception
```

Lambda-based boolean support in Scala: tests

```scala
class BooleanLambdaTest extends LambdaTest:
  import Lambda.*

  @Test
  def testNot() =
    assertEquals(Not(False), True)
    assertEquals(Not(True), False)

  @Test
  def testOr() =
    assertFalse(toBoolean(Or(False)(False)))
    assertTrue(toBoolean(Or(False)(True)))
    assertTrue(toBoolean(Or(True)(False)))
    assertTrue(toBoolean(Or(True)(True)))

  @Test
  def testAnd() =
    assertFalse(toBoolean(And(False)(False)))
    assertFalse(toBoolean(And(False)(True)))
    assertFalse(toBoolean(And(True)(False)))
    assertTrue(toBoolean(And(True)(True)))
```

Expressiveness of Lambda calculus

Multi-argument lambdas are simply encoded, essentially, by currying!

- write $\rightarrow^*$ for $\rightarrow\rightarrow \ldots \rightarrow$ (0 or many times)
- $(\lambda x_1.\lambda x_2.\ldots.\lambda x_n.L)L_1L_2\ldots L_n \rightarrow^* L\{L_1/x_1,\ldots,L_n/x_n\}$
- in Scala: `((x1,..,xn)=>L)(L1,..,Ln)`

Encoding naturals (lists and other ADTs skipped)

- $0 := \lambda s.\lambda z.z$, $1 := \lambda s.\lambda z.sz$, $2 := \lambda s.\lambda z.ssz$, $\ldots$
- $inc := \lambda n.\lambda s.\lambda z.s(nsz)$, $add := \lambda n.\lambda m.\lambda s.\lambda z.(ns(msz))$

Recursion via Y combinator

- $Y := \lambda g.(\lambda x.gxx)(\lambda x.gxx)$
- note: $YF \rightarrow^* FYF \rightarrow^* FFYF \rightarrow^* FFFYF\ldots$
- can be used to implement factorial with recursive function "$f(x) => x == 0?1 : x * f(x-1)$"

Lambda calculus is Turing-complete

Turing-completeness

- take whatever computable function f, with an encoding of its input and output domains (e.g. as naturals) in lambdas: there exists a lambda λ_f that mimicks f
- can show that λ-calculus generates all recursive computable functions
- this implies: successive, sum, product, factorial, prime number predicate, ..., any Turing-implementable function

Recursive computable functions (https://en.wikipedia.org/wiki/General_recursive_function)

A proof strategy for Turing-completeness, show you can encode

- constant functions
- successor
- projection/identity (returning an input)
- functional composition
- recursion
- minimization (find the smallest argument being the 0 of a function)

Meaning

- λ-calculus captures all is needed in terms of functions and function composition. Other FP's constructs (2-args functions, product types, sum types, ...) are just "syntactic sugar"

On evaluation order: visual sum-up

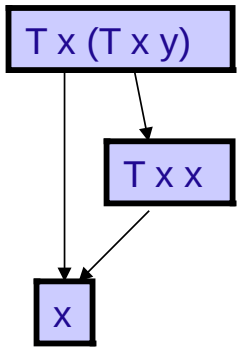
L ::= λx.L | x | LL

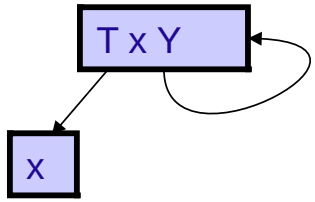
(λx.L)M → L[M/x]

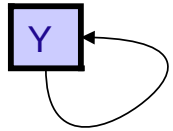
(λx.xx)(λy.(λz.yz)) → (λy.(λz.yz)) (λy.(λz.yz))

T= λt.λf.t
F= λt.λf.f
Not(x) = λx.x F T = λx.x (λt.λf.f) (λt.λf.t)
Not T→ T F T → F, Not F→ F F T → T

Y= λf. (λx.f(x x)) (λx.f(x x)), Y F → → F(Y F) ... F(F(Y F))







Confluence of Lambda calculus

All possible situations. A lambda can:

- be 1-step reduced to different lambdas, e.g.: $sum\ (inc\ 1)\ (inc\ 2)$, $T\ x\ (T\ x\ y)$
- indefinitely reduce (terminate), e.g.: $Y\ x$, Y
- terminate or not terminate, e.g.: $T\ x\ (Y)$

Lambda-calculus is confluent (Church-Rosser theorem)

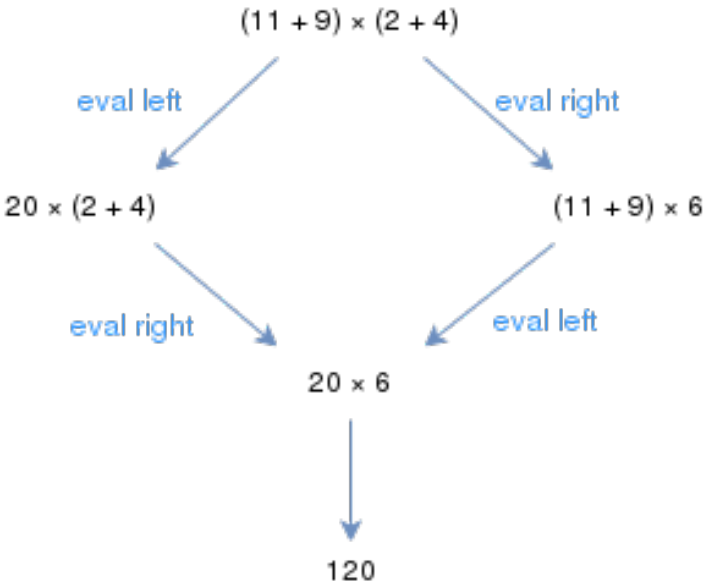
- given any L, if $L \to^* L_1$ and $L \to^* L_2$, then it exists L_3 such that $L_1 \to^* L_3$ and $L_2 \to^* L_3$, namely, so-called "don't care non-determinism"
- it is implied that no more than 1 result of computation can exist: if $L \to^* L_1 \not\to$ and $L \to^* L_2 \not\to$, then $L_1 \approx L_2$, where $L_1 \approx L_2$ means identical (modulo non-clashing renaming of variables, e.g.: $\lambda x.x \approx \lambda y.y$)

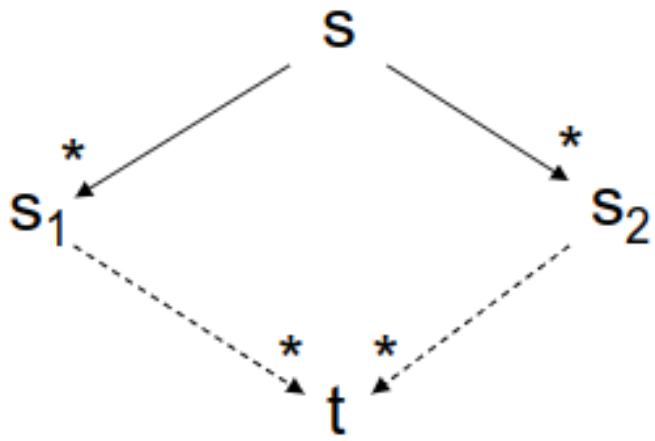
Very important meaning

- if L may not terminate, reduction choice is key to terminate (to find the result)
- if L always terminates, reduction choice is non-influential

$\Rightarrow$ Declarativity: a lambda can be taken to denote "result", achieved independently of evaluation order

Confluence


(11 + 9) × (2 + 4)
eval left
eval right
20 × (2 + 4)
(11 + 9) × 6
eval right
eval left
20 × 6
120


s
*
*
s1
s2
*
*
t

Consequences of confluence

Referential transparency

- definition: a language provides *referential transparency* if the result of evaluating an expression is independent of its context
- this allows a sort of "equational" reasoning on programs
- this facilitates understandability, refactoring, and so on
- if a program terminates, its result is independent of order of application

Declarativity

- The ability to express *what* a program computes, and not *how* this is computed
- can focus on expressing the overall result of our computation, abstracting "inner machinery"

$\Rightarrow$ recently, such low layers can enact "intelligence" which becomes optimisation/adaptiveness, often perceived as "magic", but it is just "engineering"

- declarative languages: FP and LP are the quintessential declarative paradigms

Outline

1. Generics
2. On pragmatical expressiveness
3. Declarativity and Lambda-calculus
4. Evaluation order
5. Streams
6. Extension Methods and OO-style method calls

Evaluation order in FP

Recall the issue of evaluation order

- in FP, computation may be independent of evaluation order
- this is the case with terminating computations and without side-effects, which is typically the case
  Se una computazione termina e non produce side-effects, il Teorema di Church-Rosser assicura che l'ordine di valutazione non influisce sul risultato finale.
- still, different evaluation orders can impact performance, or result if there are side-effects

Strategies for evaluation order

1. strict, call-by-value (applicative order, see Java, Scala, ...):
   ⇒ arguments evaluated just before passing them

```
def f1(a: A) = ???
```

2. non-strict, call-by-name (normal order):
   ⇒ arguments evaluated when/if used in the body

```
def f2(a: =>A) = ??? // def f2(a: ()=>A) = ???
```

3. non-strict, call-by-need (lazy evaluation, see Haskell, Miranda):
   ⇒ arguments evaluated once when/if used in the body
   viene calcolato al massimo una sola volta, la prima volta che serve all'interno del corpo, ed il suo risultato viene salvato per gli accessi successivi.

```
def f3(a: =>A) = { lazy val v = a; ??? }
```

Call-by-value, and "simulated" call-by-name/need

```
1
2   def loop[A](a: A): A = loop(a)
3
4   // standard CALL-BY-VALUE, arguments are evaluated as they are passed
5   def sPair[A](a: A, dummy: A): (A, A) = (a, a)
6   println(sPair(10, 20)) // (10,10)
7   println(sPair({println("e1");10}, {println("e2");20})) //e1 e2 (10,10)
8   println(sPair(10, loop(20))) // loop
9
10  // simulating CALL-BY-NAME, arguments are evaluated each time they occur in the body
11  def sPairSBN[A](a: () => A, dummy: () => A): (A, A) = (a(), a())
12
13  println(sPairSBN(() => 10, () => 20)) // (10,10)
14  println(sPairSBN(() => {println("e1");10}, () => {println("e2");20})) //e1 e1 (10,10)
15  println(sPairSBN(() => 10,() => loop(20))) // (10,10)
16
17  // simulating CALL-BY-NEED/LAZY, arguments are evaluated if they occur in the body
18  def sPairSL[A](a: () => A, dummy: () => A): (A, A) =
19    lazy val v = a()
20    lazy val d = dummy()
21    (v, v)
22
23  println(sPairSL(() => 10, () => 20)) // (10,10)
24  println(sPairSL(() => {println("e1");10}, () => {println("e2");20})) //e1 (10,10)
25  println(sPairSL(() => 10, () => loop(20))) // (10,10)
```

Advanced support for call-by-name and call-by-need

```scala
// supported CALL-BY-NAME, syntax =>T is a shorthand to avoid verbose 0-ary lambdas
def sPairBN[A](a: => A, dummy: => A): (A, A) = (a, a)

println(sPairBN(10, 20)) // (10,10)
println(sPairBN({println("e1");10}, {println("e2");20})) //e1 e1 (10,10)
println(sPairBN(10, loop(20))) // (10,10)

// half-simulating CALL-BY-NEED/LAZY, with lazy val and by-name args
def sPairL[A](a: => A, dummy: => A): (A, A) =
  lazy val v = a
  (v, v)

println(sPairL(10, 20)) // (10,10)
println(sPairL({println("e1");10}, {println("e2");20})) //e1 (10,10)
println(sPairL(10, loop(20))) // (10,10)
```

Supporting evaluation strategies in Scala

1 Call by-value

It is the default, and you typically do not change it

2 Call by-name

- to be chosen when execution has to be delayed, or even be avoided if not needed
- just pass a producer (0-ary lambda) instead of directly the result
- pass `()=>e` instead of `e`, and in the body use `x()` instead of `x`
- if you dislike passing `()=>e`, use syntax `=>T` instead of `()=>T`

3 Call by-need

- as of by-name, but additionally you want to prevent double evaluations for performance reasons, or due to side-effects
- you should start the body by transferring the argument to a `lazy val`
- can use syntax `=>T` as well

Outline

1. Generics
2. On pragmatical expressiveness
3. Declarativity and Lambda-calculus
4. Evaluation order
5. Streams
6. Extension Methods and OO-style method calls

An advanced example: Streams

A differenza delle liste standard che allocano tutti gli elementi in memoria al momento della creazione, uno Stream calcola ogni elemento solo quando viene esplicitamente richiesto.

Goals

- lazy management of data, up to modelling infinite flows
- streams connect by iterative decoration
- supporting iterate, map, filter, take, toList

Le operazioni su uno Stream si collegano in una catena di trasformazioni che non esegue subito il calcolo, ma prepara la "ricetta" di elaborazione.

Implemenation

- essentially, a lazy list (the name of streams in Scala 3)
- the main trick is to put lazy vals into lambdas, at cons time
- the rest of the implementation is the same of that of lists

```
1  @main def tryStreams =
2    import Streams.*
3
4    val str1 = Stream.iterate(0)(_ + 1) // {0,1,2,3,..} Creazione di uno stream infinito
5    val str2 = Stream.map(str1)(_ + 1) // {1,2,3,4,..}
6    val str3 = Stream.filter(str2)(x => (x < 3 || x > 20)) // {1,2,21,22,..}
7    val str4 = Stream.take(str3)(10) // {1,2,21,22,..,28}
8    println(Stream.toList(str4)) // [1,2,21,22,..,28] è un'operazione terminale
9
10   lazy val corec: Stream[Int] = Stream.cons(1, corec) // {1,1,1,..}
11   println(Stream.toList(Stream.take(corec)(10))) // [1,1,..,1]
```

Al momento della costruzione, la testa e la coda della lista non vengono memorizzate come valori già calcolati, ma racchiuse all'interno di lambda e lazy val. In questo modo, la coda dello Stream rimane una computazione "sospesa".

Implementation: Streams

```scala
enum Stream[A]:
  private case Empty()
  private case Cons(head: () => A, tail: () => Stream[A])

object Stream:

  def empty[A](): Stream[A] = Empty()

  def cons[A](hd: => A, tl: => Stream[A]): Stream[A] =
    lazy val head = hd
    lazy val tail = tl
    Cons(() => head, () => tail)

  def toList[A](stream: Stream[A]): Sequence[A] = stream match
    case Cons(h, t) => Sequence.Cons(h(), toList(t()))
    case _ => Sequence.Nil()

  def map[A, B](stream: Stream[A])(f: A => B): Stream[B] = stream match
    case Cons(head, tail) => cons(f(head()), map(tail())(f))
    case _ => Empty()

  def filter[A](stream: Stream[A])(pred: A => Boolean): Stream[A] = stream match
    case Cons(head, tail) if (pred(head())) => cons(head(), filter(tail())(pred))
    case Cons(head, tail) => filter(tail())(pred)
    case _ => Empty()

  def take[A](stream: Stream[A])(n: Int): Stream[A] = (stream, n) match
    case (Cons(head, tail), n) if n > 0 => cons(head(), take(tail())(n - 1))
    case _ => Empty()

  def iterate[A](init: => A)(next: A => A): Stream[A] =
    cons(init, iterate(next(init))(next))
```

Associato al enum il companion object (le varianti possono accedervi)

Outline

1. Generics
2. On pragmatical expressiveness
3. Declarativity and Lambda-calculus
4. Evaluation order
5. Streams
6. Extension Methods and OO-style method calls

Extension methods

Gli extension methods permettono di aggiungere nuovi metodi a un tipo esistente senza doverne modificare la definizione originale e senza ricorrere all'ereditarietà orientata agli oggetti

Idea

An alternative way of defining a method, extracting one of its argument: other than standard syntax for calling the method, a new one treating that argument as if it were the receiver of the invocation is enabled, but this is just syntax!

si tratta di puro zucchero sintattico: a livello di compilazione e di esecuzione non viene creato alcun oggetto o classe aggiuntiva; il compilatore traduce semplicemente la notazione a punto nella chiamata di funzione sottostante (cioè la traduce trasformando il ricevitore nel primo argomento della funzione).

Implications

- we can achieve OO-like syntax of method calls without any OOP
- we can chain method calls, e.g., to define pipelines
- a module can now define standard methods (e.g. for factories) and extension methods (e.g. for operations)

Construct

il tipo esistente da estendere con questo metodo m

```
extension (x: T) def m(arg1: T1, ..., argn: Tn): To = ...
```

Applications

- can define sequences, streams, optionals so that they can be used essentially as if we were in an OOP context

Extension methods at work

```scala
object ExtensionMethods extends App:

  // functional style for declaring operations
  def mult(n: Int, m: Int): Int = n * m

  // functional style of operation call
  println:
    mult(5, 3)

  // extension methods:
  // a twist on definition: extracting an argument
  extension (n: Int) def mult2(m: Int): Int = n * m
  // standard call in curried version is still possible
  println:
    mult2(5)(3)
  // OO-like syntax, as pure syntactic sugar
  println:
    5.mult2(3)
```

Extension methods at work on sequences

```scala
import u03.Sequences.*
import Sequence.*

// cannot chain calls!
val seq = Cons(10, Cons(20, Cons(30, Nil())))
println:
  Sequence.sum(map(filter(seq)(_ > 10))(_ + 1))

// define size and filt as extension methods on a generic Sequence[A]
extension [A](s: Sequence[A])

  def size: Int = s match
    case Cons(_, t) => 1 + t.size
    case _ => 0

  def filt(p: A => Boolean): Sequence[A] = s match
    case Cons(h, t) if p(h) => Cons(h, t.filt(p))
    case Cons(_, t) => t.filt(p)
    case _ => Nil()

// can chain method calls now!
println:
  seq.filt(_ > 10).size
```

New implementation of Sequences

```scala
object Sequences:

  enum Sequence[E]:
    case Cons(head: E, tail: Sequence[E])
    case Nil()

  object Sequence:

    extension (l: Sequence[Int])
      def sum: Int = l match
        case Cons(h, t) => h + t.sum
        case _          => 0

    extension [A](l: Sequence[A])

      def map[B](mapper: A => B): Sequence[B] = l match
        case Cons(h, t) => Cons(mapper(h), t.map(mapper))
        case Nil()      => Nil()

      def filter(pred: A => Boolean): Sequence[A] = l match
        case Cons(h, t) if pred(h) => Cons(h, t.filter(pred))
        case Cons(_, t)            => t.filter(pred)
        case Nil()                 => Nil()

    def of[A](n: Int, a: A): Sequence[A] =
      if (n == 0) then Nil[A]() else Cons(a, of(n - 1, a))
```

Il companion object si usa come modulo contenitore e factory: serve per organizzare lo spazio dei nomi e definire le funzioni di creazione e utilità generale che non agiscono su un'istanza già esistente.
Gli extension method si usano per definire le operazioni sui dati: consentono di invocare queste trasformazioni direttamente sulle istanze della sequenza sfruttando la notazione a punto in stile OO, senza dover inserire il codice dei comportamenti dentro la definizione dell'enum e mantenendo la separazione tra struttura dati e algoritmi.

Mettere gli extension method dentro il companion object è la best practice in Scala 3 per i propri tipi/ADT perché li rende disponibili "out-of-the-box" con la notazione a punto su qualsiasi istanza, senza richiedere import aggiuntivi all'utilizzatore del tipo.

Fluent use of Sequences

```scala
@main def trySequences() =
  import Sequences.*
  import Sequence.*

  val seq = Cons(10, Cons(20, Cons(30, Nil())))
  println(seq.filter(_ >= 20).map(_ + 1).sum) // 21+31 = 52
  println(sum(map(filter(seq)(_ >= 20))(_ + 1))) // equally possible
  val seq2 = of(10, -1) // Cons(-1, Cons(-1, Cons(-1, ...)))
  println(seq2.sum) // -10
```

Conclusions

On the power of FP

- very few concepts actually, and rather cleanly supported e.g. by Scala
- as a showcase, a stream library is developed with very little "boilerplate"
- note that the code of algorithms is not at all different to that of `List`